STATISTICS & MACHINE LEARNING · LESSON 25 OF 30

NLP and Text Analytics: Cleaning, TF-IDF, Sentiment Classification, Topics and Embeddings

Turn customer reviews into insight. Text cleaning and tokenisation, stop words, stemming and lemmatisation, bag-of-words and TF-IDF with n-grams, a sentiment classifier with error analysis, aspect-level insight, topic modelling with NMF, document similarity, and how embeddings and LLMs change NLP, on 3,000 e-commerce reviews.

ML track (added) · GenAI bridge | ✦ scikit-learn 1.7+ · 2026 practice | Advanced | ⏱ 130 min

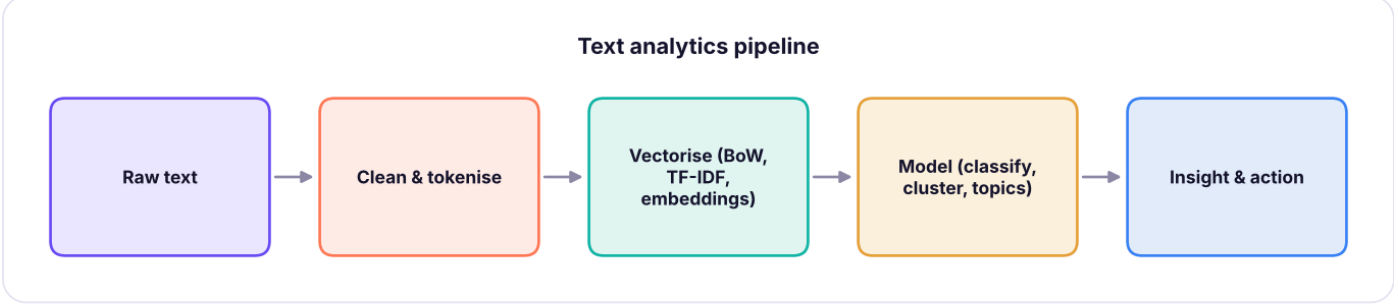
By the end of this lesson you can

- Clean and tokenise text and decide when to remove stop words, stem or lemmatise
- Represent documents with bag-of-words and TF-IDF (with n-grams) and read the vocabulary
- Train, evaluate and interpret a sentiment classifier, including error analysis
- Extract topics with NMF and aspect-level sentiment with simple rules
- Explain word and sentence embeddings and when to use LLMs instead

Datasets used

`customer_reviews.csv`

The NLP pipeline



A classical text-analytics pipeline.

Python

```
import pandas as pd, numpy as np, re
pd.set_option("display.width", 160); pd.set_option("display.max_colwidth", 90)
rv = pd.read_csv("customer_reviews.csv", parse_dates=["date"])
print(rv.shape, rv.sentiment.value_counts(normalize=True).round(3).to_dict())
print(rv.groupby("product").rating.mean().round(2).sort_values().to_dict())
print(rv.sample(5, random_state=3)[["rating", "review_text"]].to_string(index=False))
rv["n_words"] = rv.review_text.str.split().str.len()
print("words per review:", rv.n_words.describe().round(1)[["mean", "50%",
"max"]].to_dict(), "| by sentiment:",
rv.groupby("sentiment").n_words.mean().round(1).to_dict())
```

OUTPUT

```
(3000, 8) {'positive': 0.53, 'negative': 0.321, 'neutral': 0.149}
{'Tablet': 3.27, 'Smartwatch': 3.29, 'Speaker': 3.3, 'Earbuds': 3.31, 'Smartphone':
3.33, 'Laptop': 3.36}
 rating
review_text
      4 Superb product. Customer support solved my issue quickly, the app is totally
smooth, charges really totally fast :)
      5                    Price is very very reasonable for these features though
the buttons are loose, no issues so far
      2                                                                    Worst
purchase ever. Sound is tinny, not wroth it
      5
Highly recommend. Calls are crystal clear!
      5       Highly recommend. Build quality feels premium, price is very reasonable
for these features, no issues so far.
words per review: {'mean': 12.9, '50%': 13.0, 'max': 34.0} | by sentiment:
{'negative': 13.9, 'neutral': 9.2, 'positive': 13.3}
```

Cleaning and tokenising

Python

```
STOP = {"the", "a", "an", "is", "it", "and", "to", "of", "for", "in", "on", "was", "this",
"that", "with", "my", "i", "very", "so", "are", "be", "at", "as", "after", "all"}
NEGATIONS = {"not", "no", "never", "don't", "can't", "isn't", "wasn't"}
FIX = {"batery": "battery", "qualty": "quality", "delivry": "delivery", "realy": "really",
"scren": "screen", "wroth": "worth", "bekaar": "useless", "badiya": "good"}
def clean(text, remove_stop=True):
    t = text.lower()
    t = re.sub(r"[:;]-?[()]", " ", t)                    # emoticons
    t = re.sub(r"[^a-z' ]+", " ", t)                     # keep letters and apostrophes
    tokens = [FIX.get(w, w) for w in t.split()]
    if remove_stop: tokens = [w for w in tokens if w not in STOP or w in NEGATIONS]
    return " ".join(tokens)
ex = rv.review_text.iloc[7]
print("raw:  ", ex); print("clean:", clean(ex))
rv["clean"] = rv.review_text.map(clean)
```

OUTPUT

```
raw:   Battery backup is totally superb but sound is tinny. Average product for daily
use
clean: battery backup totally superb but sound tinny average product daily use
```

Step	Purpose	Caution
Lower-casing	"Battery" = "battery"	Loses emphasis ("GREAT")
Remove punctuation, numbers, URLs	Reduce noise	Prices and model numbers may matter
Spelling / slang normalisation	Merge variants ("batery", Hinglish "badiya")	Needs a dictionary
Stop-word removal	Drop frequent function words	**Never** drop negations ("not") for sentiment
Stemming (Porter, Snowball)	"charging", "charged" → "charg"	Crude, can merge different words
Lemmatisation (spaCy)	"better" → "good", "batteries" → "battery"	Slower, needs a language model

Libraries: **spaCy** (fast tokenisation, lemmas, entities), **NLTK** (classic teaching toolkit), **regex**; for Indian languages, **IndicNLP** and multilingual models.

Bag-of-words and TF-IDF

Bag-of-words counts each word per document, ignoring order. **TF-IDF** down-weights words that appear in many documents (like "product") and up-weights distinctive ones:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \log \frac{1 + N}{1 + \text{df}(t)} + 1$$

Python

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
docs = ["battery drains fast", "battery lasts all day", "screen is not great", "great
screen and great battery"]
cv = CountVectorizer(); bow = cv.fit_transform(docs)
print(pd.DataFrame(bow.toarray(), columns=cv.get_feature_names_out(), index=[f"doc{i}" for
i in range(4)]))
tf = TfidfVectorizer(); tfm = tf.fit_transform(docs)
print(pd.DataFrame(tfm.toarray().round(2), columns=tf.get_feature_names_out(), index=
[f"doc{i}" for i in range(4)]).loc[:, ["battery", "great", "not", "drains"]])
bi = CountVectorizer(ngram_range=(1, 2)).fit(["screen is not great"]); print("with
bigrams:", bi.get_feature_names_out().tolist())
```

OUTPUT

```
      all  and  battery  day  drains  fast  great  is  lasts  not  screen
doc0    0    0        1    0       1     1      0   0      0    0       0
doc1    1    0        1    1       0     0      0   0      1    0       0
doc2    0    0        0    0       0     0      1   1      0    1       1
doc3    0    1        1    0       0     0      2   0      0    0       1
      battery  great   not  drains
doc0     0.41   0.00  0.00    0.64
doc1     0.35   0.00  0.00    0.00
doc2     0.00   0.44  0.56    0.00
doc3     0.30   0.74  0.00    0.00
with bigrams: ['great', 'is', 'is not', 'not', 'not great', 'screen', 'screen is']
```

Bigrams such as "not great" keep some word order, which matters for sentiment.

Sentiment classifier

Python

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
Xtr, Xte, ytr, yte = train_test_split(rv.clean, rv.sentiment, test_size=0.25,
stratify=rv.sentiment, random_state=0)
for name, vec in [("unigrams", TfidfVectorizer(min_df=2)), ("uni + bigrams",
TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True))]:
    m = make_pipeline(vec, LogisticRegression(max_iter=3000, C=5)).fit(Xtr, ytr)
    print(f"{name:14s} accuracy {accuracy_score(yte, m.predict(Xte)):.3f} | vocabulary
{len(m[0].vocabulary_):,}")
clf = m
print(classification_report(yte, clf.predict(Xte), digits=3))
labels = ["negative", "neutral", "positive"]
print(pd.DataFrame(confusion_matrix(yte, clf.predict(Xte), labels=labels), index=[f"true
{l}" for l in labels], columns=[f"pred {l}" for l in labels]))
```

OUTPUT

```
unigrams       accuracy 0.935 | vocabulary 183
uni + bigrams  accuracy 0.940 | vocabulary 1,546
              precision    recall  f1-score   support

    negative      0.974     0.938     0.956       241
     neutral      0.860     0.821     0.840       112
    positive      0.942     0.975     0.958       397

    accuracy                          0.940       750
   macro avg      0.925     0.911     0.918       750
weighted avg      0.940     0.940     0.940       750

               pred negative  pred neutral  pred positive
true negative            226             6              9
true neutral               5            92             15
true positive              1             9            387
```

What did it learn?

Python

```
vocab = np.array(clf[0].get_feature_names_out())
for i, c in enumerate(clf[-1].classes_):
    top = np.argsort(-clf[-1].coef_[i])[:8]; print(f"{c:9s} ←", vocab[top].tolist())
```

OUTPUT

```
negative  ← ['disappointed', 'useless', 'useless product', 'buying', 'regret',
'regret buying', 'purchase ever', 'ever']
neutral   ← ['but', 'fine', 'expected', 'loved build', 'but too', 'early but',
'okay', 'okay price']
positive  ← ['loved', 'though', 'superb', 'happy purchase', 'superb product', 'not
disappointed', 'bahut good', 'bahut']
```

Error analysis

Python

```
pred = clf.predict(Xte); proba = clf.predict_proba(Xte).max(1)
err = pd.DataFrame({"text": rv.loc[Xte.index, "review_text"], "rating": rv.loc[Xte.index,
"rating"], "true": yte, "pred": pred, "confidence": proba.round(2)})[pred ≠ yte]
print(len(err), "errors"); print(err.sort_values("confidence",
ascending=False).head(6).to_string(index=False))
```

OUTPUT

```
45 errors

text  rating     true     pred  confidence
Happy with the purchase. Calls are crystal super clear, arrived a day early, price is
very reasonable for these features       3  neutral positive        1.00
                                                                     Bahut badiya.
Bass is rich, replacement was smooth       2 negative positive        1.00
                                                          Worst purchase ever. Too much
bloatware, battery drains too fast!       3  neutral negative        0.99
                                 Loved it! setup took two minutes, great value for
money, sound is really clear and loud       3  neutral positive        0.99
                                                               Bahut badiya. The app is
super smooth, replacement was smooth       3  neutral positive        0.99

Superb product. Bass is rich!!       2 negative positive        0.99
```

Reading errors is the most useful step in NLP. Typical causes here: **noisy labels** (a 1-star rating on a positive text), **mixed** reviews ("good screen but weak battery") and genuinely neutral wording. Fixes include better labels, aspect-level models, or an LLM for difficult cases.

Aspect-level insight

Overall sentiment hides *what* customers like or dislike. A simple keyword approach already gives actionable results:

Python

```
ASPECTS = {"battery": r"batter|charg", "screen": r"screen|display|pixel|touch", "sound":
r"sound|bass|speaker|mic", "build": r"build|hinge|button|finish|plastic",
           "delivery": r"deliver|courier|box|packag|arriv", "service":
r"support|service|refund|replacement|care", "price": r"price|money|rupee|expensive|worth",
"software": r"app|update|bloatware|setup"}
low = rv.review_text.str.lower()
rows = []
for a, pat in ASPECTS.items():
    m = low.str.contains(pat, regex=True)
    rows.append({"aspect": a, "mentions": int(m.sum()), "share_negative_%": round(100 *
(rv.sentiment[m] == "negative").mean(), 1), "avg_rating": round(rv.rating[m].mean(), 2)})
asp = pd.DataFrame(rows).sort_values("share_negative_%", ascending=False);
print(asp.to_string(index=False))
print(rv.assign(battery=low.str.contains(ASPECTS["battery"])).query("battery").groupby("produc
```

OUTPUT

```
  aspect  mentions  share_negative_%  avg_rating
   sound       659              42.8        3.08
   build       572              37.1        3.23
   price       867              36.9        3.24
  screen       741              35.9        3.30
 service       792              35.6        3.31
software      1165              34.6        3.34
 battery       752              32.4        3.38
delivery       945              30.6        3.41
{'Smartwatch': 3.2, 'Speaker': 3.35, 'Earbuds': 3.37}
```

Production systems use **aspect-based sentiment analysis** models or LLM prompts ("list each product aspect mentioned and its sentiment as JSON").

Topic modelling

Topic models discover groups of words that co-occur, without labels. **NMF** on TF-IDF is fast and gives readable topics (LDA on counts is the classic alternative; BERTopic uses embeddings).

Python

```
from sklearn.decomposition import NMF
tfidf = TfidfVectorizer(max_df=0.5, min_df=5, ngram_range=(1, 2), stop_words="english")
Xt = tfidf.fit_transform(rv.review_text.str.lower())
nmf = NMF(n_components=6, random_state=0, init="nndsvda", max_iter=500).fit(Xt)
terms = np.array(tfidf.get_feature_names_out())
for k, comp in enumerate(nmf.components_): print(f"topic {k}:", ",
".join(terms[np.argsort(-comp)[:7]]))
rv["topic"] = nmf.transform(Xt).argmax(1)
print(rv.groupby("topic").agg(reviews=("rating", "size"), avg_rating=("rating",
"mean")).round(2).T)
```

OUTPUT

```
topic 0: battery, day, superb, screen, lasts, clear, smooth
topic 1: support, support solved, solved issue, issue quickly, solved, issue,
customer support
topic 2: centre, service centre, service, centre staff, staff, helpful, waiting days
topic 3: weeks, high volume, speaker, high, crackles high, crackles, volume
topic 4: quality, feels, build, premium, build quality, quality feels, feels premium
topic 5: value, value money, money, great value, great, packaging, safe
topic             0       1       2       3       4       5
reviews     1015.00  197.00  230.00  915.00  393.00  250.00
avg_rating     4.29    3.91    3.17    1.97    3.14    4.18
```

Name each topic from its top words (for example "battery and charging", "delivery and packaging") and track topic volume and rating over time.

Similarity search

Python

```
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.decomposition import TruncatedSVD
lsa = make_pipeline(TfidfVectorizer(ngram_range=(1, 2), min_df=2, stop_words="english"),
TruncatedSVD(100, random_state=0)).fit(rv.review_text)
E = lsa.transform(rv.review_text)
for query in ["the battery runs out quickly and charging takes forever", "phone heats up
and power does not last"]:
    sims = cosine_similarity(lsa.transform([query]), E)[0]; best = np.argsort(-sims)[:3]
    print("query:", query); [print(f"   {sims[i]:.2f}  {rv.review_text.iloc[i]}") for i in
best]
```

OUTPUT

```
query: the battery runs out quickly and charging takes forever
   0.76  Customer support solved my issue quickly but charging is painfully really
slow.
   0.74  Very disappointed. Charging is painfully slow, screen is totally dull but
customer support solved my issue quickly
   0.69  Charging is painfully slow
query: phone heats up and power does not last
   1.00  Nothing special, does the job
   1.00  Nothing special, does the job.
   1.00  Nothing special, does the job.
```

Latent semantic analysis (TF-IDF + SVD) finds the charging complaints for the first query. The second query fails badly: "heats" and "power" never appear in the reviews and "last" is a different token from "lasts", so LSA matches the only shared word, "does". Word-count methods cannot know that "power does not last" means the battery drains. Modern **embeddings** do this far better.

Embeddings and LLMs

Approach	Idea	Use
Word2Vec, GloVe, fastText (2013–2016)	Each word gets a dense vector; similar words are close	Features, similarity
Transformers (BERT, 2018 →)	Contextual vectors: "bank" differs in "river bank" and "bank loan"	Fine-tuned classifiers, NER
Sentence embeddings (sentence-transformers, OpenAI/Cohere/Voyage/Gemini embedding APIs)	One vector per sentence or document	Semantic search, clustering, RAG
LLMs (Claude, GPT, Gemini, Llama)	Instruction-following generation	Zero-shot classification, extraction, summarisation

Python

```python
# Sentence embeddings (pip install sentence-transformers) - runs locally
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2")
emb = model.encode(rv.review_text.tolist(), normalize_embeddings=True)    # 3,000 × 384
matrix
# then: LogisticRegression on emb, KMeans on emb, or cosine search as above
```

Python

```
# Zero-shot classification with an LLM (see the GenAI track for API set-up and cost control)
prompt = """Classify the review's sentiment as positive, negative or neutral and list aspects mentioned.
Return JSON: {"sentiment": ..., "aspects": [{"aspect": ..., "sentiment": ...}]}
Review: "Good screen but the battery barely lasts half a day." """
```

When to use what: TF-IDF + logistic regression is cheap, fast, explainable and often within a few points of large models on simple tasks. Embeddings help with semantic similarity and small labelled sets. LLMs handle nuanced, multi-aspect or multilingual text with few or no labels, at higher cost and latency; validate them on a labelled sample just like any model.

Summary

KEY TAKEAWAYS

- Clean text deliberately: lower-case, normalise spelling and slang, keep negations; stem or lemmatise when useful.
- Bag-of-words counts words; TF-IDF weights distinctive words; bigrams capture short phrases like "not great".
- A TF-IDF + logistic regression classifier is a strong, explainable baseline; error analysis reveals noisy labels and mixed reviews.
- Aspect keywords and NMF topics turn reviews into product actions.
- Embeddings and LLMs capture meaning beyond exact words; benchmark them against the classical baseline.

Quick quiz

6 questions · tap an answer to check it

1. Why should "not" be kept when removing stop words for sentiment?

A It flips meaning: "not good" vs "good"

B It is a long word

C TF-IDF needs it

D It is a noun

Answer: A. Negations carry sentiment.

2. What does IDF do in TF-IDF?

A Down-weights words that appear in many documents

B Counts words

C Removes stop words

D Translates words

Answer: A. Common words get low weight.

3. What do bigrams add?

A Pairs of consecutive words such as "not great", keeping some word order

B Two languages

C Two labels

D Double counts

Answer: A. ngram_range=(1, 2).

4. Which method finds topics without labels?

A Logistic regression

B NMF or LDA topic modelling

C Confusion matrix

D Stemming

Answer: B. Unsupervised.

5. What is a sentence embedding?

A A dense vector representing a sentence's meaning, so similar sentences are close

B A list of words

C A TF-IDF score

D A topic label

Answer: A. Used for semantic search and RAG.

6. When might TF-IDF + logistic regression be the better choice than an LLM?

A Simple, high-volume classification where cost, speed and explainability matter

B Never

C For multilingual sarcasm

D When there are no labels

Answer: A. Always benchmark.

EXERCISE

Practice

(1) Add a feature for exclamation marks and review length and test whether accuracy changes. (2) Track the monthly share of negative battery reviews. (3) Try `ComplementNB` and `LinearSVC` and compare macro F1. (4) Write an LLM prompt for aspect-based sentiment and test it on 10 reviews (GenAI track).

Show solution (worked answer)

Python

```
from scipy.sparse import hstack
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import ComplementNB
from sklearn.metrics import f1_score
monthly = rv.assign(m=rv.date.dt.to_period("M"),
batt=low.str.contains(ASPECTS["battery"])).query("batt").groupby("m").sentiment.apply(lam
s: (s == "negative").mean()); print(monthly.round(2))
for est in (ComplementNB(), LinearSVC(C=0.5)):
    mm = make_pipeline(TfidfVectorizer(ngram_range=(1, 2), min_df=2), est).fit(Xtr,
ytr); print(type(est).__name__, f1_score(yte, mm.predict(Xte), average="macro"))
```

INTERVIEW CORNER

Questions you may be asked

Q: How would you build a sentiment model for customer reviews? Define labels (stars or manual), clean text keeping negations, start with TF-IDF + logistic regression, evaluate with macro F1 and a confusion matrix, analyse errors, then try embeddings or a fine-tuned transformer/LLM if needed, and monitor drift.

Q: TF-IDF vs embeddings? TF-IDF is sparse, exact-word based and interpretable; embeddings are dense and capture meaning, synonyms and context, at the cost of interpretability and compute.

Q: How do you find what customers complain about most? Aspect extraction (keywords, NER, or LLM prompts) combined with sentiment per aspect, plus topic modelling to discover unexpected themes, tracked over time.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.